# An Optimal Deterministic Adversary for Sorting

Nicholas Nash
ORCID: 0009-0003-0475-7852

8 November 2026

**Abstract**

This paper describes an optimal *deterministic*, polynomial-time adversary for ordinary sorting. For any correct sorting algorithm, the adversary forces that algorithm to make $(1 - o(1))n \log_2 n$ comparisons. The best previously known adversary forced $\frac{37}{64} n \log_2 n$ comparisons. The adversary retains comparisons in directed acyclic graphs and represents possible orders by a region of real positions satisfying interval bounds and the recorded inequalities. It estimates the fraction of this region's volume retained by each answer through deterministic calculations on bounded graph neighborhoods. Moving items into ordered subproblems controls graph degrees and directed path lengths; a geometric analysis shows that these moves preserve the optimal asymptotic bound.

## Methodological disclosure

Except for the abstract, the text of this paper is AI-generated using GPT 6 Astra. The author is working on a clear, human-readable exposition.

## 1 Introduction

We consider comparison sorting of $n$ distinct labelled keys. A sorter chooses each pair using the answers to its earlier comparisons and must be correct for every strict total order on the labels. A sorting adversary supplies those answers as the comparisons arrive. It is *legal* if every finite sequence of its answers is consistent with at least one total order. The adversary knows $n$ and sees the comparisons, but need not inspect the sorter that generates them. Throughout, log denotes $\log_2$, and ln the natural logarithm; we assume $n \geq 2$.

The classical sorting lower bound guarantees the existence of a hard input for every correct sorter. Here the problem is to produce hard answers by one *deterministic, polynomial-time* procedure. The polynomial bound applies to the adversary's computation; the comparison lower bound applies to the sorter. Without the time restriction, a deterministic optimal adversary is immediate. With randomness, polynomial-time adversaries achieve the optimal asymptotic bound in expectation and with high probability.

**Without the time restriction.** Keep all total orders consistent with the answers so far. Each comparison partitions them into two classes; answer so as to retain a larger class. After $q$ comparisons, at least $n!/2^q$ orders remain, forcing $q \geq \log(n!)$. This rule is deterministic, but enumerating its classes may require factorial work. A consistent total order is a *linear extension* of the partial order given by the answered comparisons; counting these extensions exactly is #P-complete [8].

**Deterministic adversaries from graph entropy.** Kahn and Kim [10] already give a deterministic polynomial-time adversary for sorting with partial information. Starting from any given partial order $P$, it answers comparisons online and forces $\Omega(\log e(P))$ comparisons, where $e(P)$ is the number of linear extensions of $P$. Their method uses graph entropy and convex optimization via the ellipsoid method. For ordinary sorting, the initial partial order has no relations, so $e(P) = n!$ and their guarantee is $\Omega(n \log n)$. Thus deterministic polynomial-time adversaries are available even for arbitrary initial partial information. The remaining question for ordinary sorting is whether such an adversary can force $(1 - o(1))n \log n$ comparisons.

**With randomness.** An efficient adversary can simply choose a uniformly random permutation of the labels, keep it hidden, and answer from the ranks in that permutation. For every fixed correct sorter, the usual comparison-tree argument gives an expected cost of at least $\log(n!)$. It also gives a high-probability bound. At most $2^q$ permutations can terminate at leaves of depth at most an integer $q$, so the comparison count $C$ satisfies

$$\mathbb{P}(C \le q) \le \frac{2^q}{n!}.$$

Thus $C \ge \log(n!) - O(\log n)$ with probability $1 - o(1)$. Generating an exactly uniform permutation takes expected polynomial bit time; subsequent answers require only rank lookups. A fixed polynomial time bound is obtained by using an almost uniform permutation, with a vanishing additional failure probability. These guarantees also hold for an always-correct randomized sorter whose random bits are independent of the adversary's: condition on the sorter's random choices and apply the same argument.

For any prescribed permutation, a correct sorter can first check its $n - 1$ consecutive pairs, finish if all checks agree, and otherwise use a general sorting algorithm. That permutation then costs only $n - 1$ comparisons. The deterministic adversary must choose its answers in response to the questions it receives.

**Randomized approximation of the number of linear extensions.** Randomization also makes it possible to estimate how many linear extensions remain after each possible answer. Given the current partial order $P$, assign each label a real coordinate in $(0, 1)$ and form the region

$$K(P) = \{X \in (0,1)^n : X_i < X_j \text{ whenever } i <_P j\}.$$

Each allowed total order occupies volume $1/n!$ in the unit cube. Hence $\mathrm{vol}(K(P)) = e(P)/n!$ [7]. The two possible answers to a comparison cut this region into two parts, and their volumes are proportional to their counts of allowed orders.

Dyer, Frieze, and Kannan [9] give a randomized polynomial-time algorithm for relative volume approximation and explicitly note its application to linear extensions. To use it here, first check whether the queried relation is already forced by $P$. If so, return it directly. Otherwise both directions are feasible and give positive-volume regions. Estimate their volumes and choose the larger estimate. When both estimates have relative error at most $\varepsilon$, the chosen part retains at least $(1 - \varepsilon)/2$ of the current volume. The counting argument then forces

$$q \ge \frac{\log(n!)}{1 - \log(1 - \varepsilon)} = (1 - O(\varepsilon)) \log(n!)$$

comparisons. Taking $\varepsilon = 1/\lceil \log(n + 1) \rceil$ gives $n \log n - O(n)$.

For a high-probability guarantee, allow $Q = 2n \lceil \log(n + 1) \rceil$ queries before fixing a consistent order. Amplify each volume estimate to failure probability at most $n^{-4}$. Each call uses fresh randomness, so this bound holds conditional on the preceding history. A union bound over at most $2Q$ estimates gives total failure probability $O(\log n / n^3)$, even for adaptive queries. These

accuracy requirements preserve polynomial time. Every run remains legal, including when an estimate fails, because only feasible answers are offered.

The issue is therefore *derandomization*: obtain the optimal asymptotic comparison bound with a single deterministic response rule whose work per comparison is polynomial in $n$. Kaplan, Zamir, and Zwick [5, Section 7] explicitly ask for an adversary forcing $(1 - o(1))n \log n$ comparisons. Our main theorem answers this question.

**Theorem 1.1.** *There is a deterministic legal adversary, initialized using only $n$, that performs a fixed polynomial number of bit operations per comparison and forces every correct comparison sorter to make*

$$(1 - o(1))n \log n$$

*comparisons.*

Merge sort uses at most $n \lceil \log n \rceil$ comparisons, so this bound is asymptotically optimal. The running-time guarantee is asymptotic and involves very large parameter thresholds.

Section 2 reviews the earlier adversaries. Section 3 gives our data structure and comparison procedure. Items are stored at nodes of a rooted tree, with a directed acyclic graph recording comparisons within each node. Interval bounds and the graph's inequalities define a region of real positions representing the possible orders there. The adversary computes lower bounds on the fraction of this region's volume retained by each answer. Three technical ingredients make this possible.

*Certifying comparisons with enclosing intervals.* After propagating interval bounds through the full graph, we perform finite computations on a bounded neighborhood. Each computation returns intervals enclosing uniformly distributed feasible positions. Disjoint intervals establish a comparison, so counting these cases gives certified lower bounds on the probabilities of the two answers. Degree and path-length bounds ensure that almost all returned intervals are short; density bounds then show that few comparisons remain unresolved (Section 6).

*Choosing positions with large remaining volume.* Moving an item into a child fixes its position in the parent. We choose a binary fraction with the fewest digits in a suitable interval. For a wide coordinate, the finite calculation certifies substantial probability inside this interval and on both sides. Log-concavity then gives every point inside a density comparable to the peak. For a narrow coordinate, the middle half of its allowed interval supplies the required volume gain (Section 5.3).

*Bounding the cost of graph simplification.* Section 4 compares the volume gained by moving items with the probability of choosing their children in a counting experiment. Items confined to very short intervals and groups of items from long directed paths supply enough gain to pay for these choices. Moves triggered by high degree are charged to the comparisons whose arcs they remove. The experiment gives a lower bound on the number of orders represented by any state, connecting the analysis to the sorter's task of determining one order.

Finally, Section 7 lets the degree and path-length bounds grow slowly with $n$, keeping the computation polynomial as the comparison coefficient approaches one.

Appendix A proves the density estimates, Appendix B proves the local enclosure theorem, Appendix C verifies the explicit comparison coefficient, and Appendix D collects the parameters. The manuscript source is available online.[1]

[1] https://github.com/nicknash/sorting-adversary-proofs/tree/main/for-humans

# 2 Related Work

## 2.1 Binary-tree adversaries

Atallah and Kosaraju [1] gave an efficient sorting adversary with a lower bound of $\frac{1}{2}n \log n$ comparisons. Related adversaries occur in the work of Borodin, Guibas, Lynch, and Yao [2] on searching after partial sorting, Richards and Vaidya [3] on the distribution of comparisons among keys, and Brodal, Chaudhuri, and Radhakrishnan [4] on maintaining the minimum. The binary-tree rule of Borodin et al. and Brodal et al. provides the starting point for our construction.

Place every item at the root of an ordered binary tree. Items in disjoint subtrees have the left-to-right order of those subtrees. A comparison changes the placement as follows.

- If both items occupy the same node, move one to its left child and the other to its right child, and answer accordingly.
- If one item is at an ancestor of the other, move the higher item to the child whose subtree does not contain the lower item.
- If their nodes are unrelated by ancestry, answer from their left-to-right order, without moving either item.

For a concrete example, start with $A, B, C, D$ at the root. Answering $A < B$ moves $A$ to the left child and $B$ to the right, while $C, D$ remain at the root. If the sorter next compares $A$ and $C$, the ancestor rule moves $C$ to the right child and gives $A < C$. Comparing $D$ and $B$ then moves $D$ to the left child and gives $D < B$. The two children now contain $\{A, D\}$ and $\{B, C\}$. A subsequent comparison of $D$ with $C$ is answered $D < C$ without moving either item. The orders within $\{A, D\}$ and within $\{B, C\}$ remain unresolved. Figure 1 shows the placements.

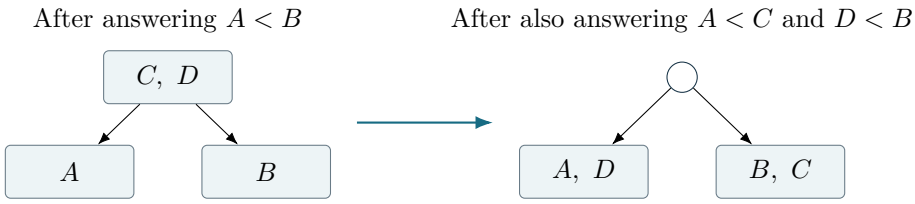



**Figure 1:** The binary-tree adversary. The first comparison separates two items at the same node. The next two comparisons each move an item from an ancestor into the opposite child. In the final state, comparing $D$ with $C$ requires no movement.

Each comparison increases the sum of item depths by at most two. At completion, occupied nodes must be distinct and unrelated by ancestry: otherwise two represented items can occur in either relative order, leaving an unresolved comparison. Associate the root with $(0, 1)$, and associate each child with the corresponding half of its parent's interval. A node of depth $d_i$ therefore represents an interval of length $2^{-d_i}$. At completion these intervals are disjoint, giving

$$\sum_i 2^{-d_i} \le 1, \qquad 2^{-\frac{1}{n}\sum_i d_i} \le \frac{1}{n}\sum_i 2^{-d_i} \le \frac{1}{n}.$$

Convexity gives the bound on $2^{-\frac{1}{n}\sum_i d_i}$. Hence $\sum_i d_i \ge n \log n$, and at least $\frac{1}{2}n \log n$ comparisons are necessary [4, Section 4.1].

The tree representation can settle relations that have never been queried. For example, start again with four items at the root and answer $A < B$ and $C < D$. Placing $A, C$ in the left child and $B, D$ in the right child also commits the adversary to $A < D$ and $C < B$. Neither relation follows from those two answers alone.

## 2.2 The adversary of Kaplan, Zamir, and Zwick

Kaplan, Zamir, and Zwick [6] replace each step of a four-child tree by a small directed acyclic graph (DAG), allowing an item to move towards a group of consecutive children before selecting one child. Number the children $0, 1, 2, 3$. The intermediate node $[i, j]$ represents the possible destinations $i, i+1, \ldots, j$. The parent itself is $[0, 3]$, and $[i, i]$ is child $i$. For $i < j$, a downward step removes one endpoint of the range, reaching $[i, j-1]$ or $[i+1, j]$. Figure 2 shows this interval lattice. For example, $[0, 2]$ and $[1, 3]$ both lead to $[1, 2]$; the intermediate nodes have shared descendants. The same construction is repeated at every node of the four-child tree.

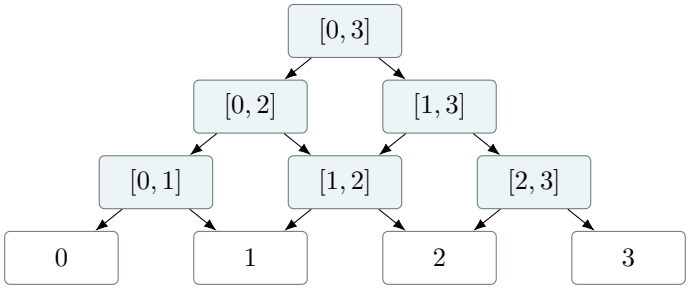



**Figure 2:** One step of the four-child tree in the adversary of Kaplan, Zamir, and Zwick [6, Section 5]. An intermediate node records a range of possible children. In particular, an item at $[1, 3]$ has not committed to child 1, 2, or 3.

Within a node, the adversary stores unpaired items and disjoint ordered pairs. There can be many items at a node, but its retained comparison components have at most two vertices. Tree nodes and intermediate nodes representing three children may store pairs; intermediate nodes representing two children store only unpaired items. Comparing two unpaired items $A, B$ at a tree node can therefore record $A < B$ as a pair without moving either item.

A comparison between pairs in different nodes illustrates the role of the lattice. Suppose $A < B$ is retained at $[0, 2]$ and $C < D$ at $[1, 3]$. On comparing $A$ and $C$, one permitted response is $A < C$: move $A$ into child 0 and $B$ into $[1, 2]$, leaving the pair $C < D$ at $[1, 3]$. Figure 3 shows the change. The old answer $A < B$ is now represented by the items' positions, while $C < D$ remains a retained pair [6, Table 11].

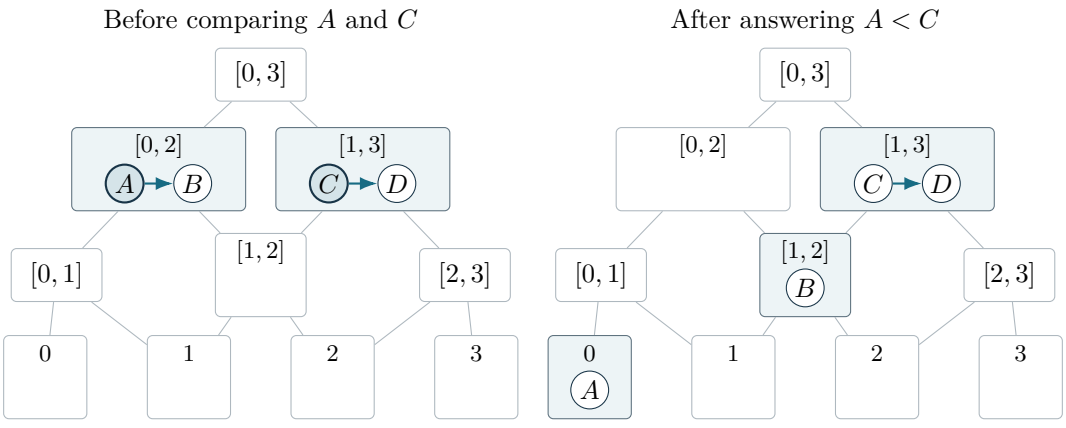



**Figure 3:** A comparison between pairs at different nodes in the adversary of Kaplan, Zamir, and Zwick [6, Table 11]. Shaded items are queried. Blue arcs are retained comparisons; gray lines are lattice edges. The new answer $A < C$ is enforced by the disjoint child ranges assigned to $A$ and $C$.

A retained pair also restricts its members' possible destinations: they cannot enter the same child of the underlying four-child tree. For example, if $A < B$ is stored at $[0, 2]$, then $A$ can enter only child 0 or 1, and $B$ can enter only child 1 or 2. The pair's order thus narrows

each member's possible destinations, and the adversary uses these restrictions when choosing subsequent comparison answers [6, Section 5].

Their analysis measures progress using item depths, intermediate placements, and retained pairs. The comparison rules bound the increase in this measure and give a lower bound of $\frac{37}{64}n \log n$ comparisons, improving the earlier $\frac{1}{2}n \log n$ bound. Their concluding discussion proposes allowing more complicated comparison components than pairs, and asks whether an efficient deterministic adversary can force $(1 - o(1))n \log n$ comparisons [6, Section 6].

# 3 The adversary

We describe the data structure maintained by our adversary and how it is updated in response to comparison queries from the sorting algorithm.

Kaplan, Zamir, and Zwick [6] retain disjoint ordered pairs within a node. Our adversary maintains a rooted tree with a DAG at each node, so several retained comparisons can involve the same item. For example, it can keep $a < b$ and $a < c$ together while leaving the order of $b$ and $c$ undecided. This lets it represent richer partial orders before moving items further down the tree.

The computational challenge is to use these richer orders without having to approximately count the linear extensions of an entire DAG. We periodically simplify a node's DAG by removing vertices and moving their items into child nodes, creating children when needed. Constraints at the parent preserve the comparisons involving the moved items, while their order within a child can remain unresolved. These simplifications allow comparison answers to be chosen by deterministic calculations on small neighborhoods in that node's DAG. The analysis must show that the resulting loss of possible orders is small enough to retain the optimal comparison bound.

## 3.1 The data structure

Our data structure is a rooted tree. Each node stores item records and a single directed acyclic graph (DAG), whose vertices are the items recorded at that node. We call this its *in-node DAG*. Figure 4 shows an example.

Each item has a record $(x, X_x, a_x, b_x)$ stored at exactly one node. The label $x$ identifies its key, and $X_x$ is a symbolic variable for its position among the items at that node. The fields $a_x, b_x$ restrict this variable to the open interval $(a_x, b_x) \subseteq (0, 1)$. The bounds $a_x, b_x$ are fractions whose denominators are powers of two; we call such numbers *dyadic rationals*. We call an item *free* at its node while its position remains variable.

In an in-node DAG, an arc $x \to y$ records a comparison answered $x < y$ between two items stored there, imposing $X_x < X_y$ on their positions. Each answered comparison between free items contributes at most one arc.

A *feasible assignment* at a node is a choice of real values for its position variables satisfying $a_x < X_x < b_x$ for every item $x$ and $X_x < X_y$ for every arc $x \to y$ in its in-node DAG.

Initially the tree consists of just the root, containing all records with bounds $a_x = 0$ and $b_x = 1$. The position variables $X_x$ are unassigned, and the root's in-node DAG has no arcs. Thus every assignment in $(0, 1)^n$ is feasible, and every total order can be obtained by assigning distinct positions.

Each child is attached to a fixed position $c \in (0, 1)$ in its parent, called its *anchor*. A node has at most one child at each anchor. For any feasible assignment at the parent, an item with position below $c$ precedes every item in that child's subtree; an item with position above $c$ follows every item in it. Subtrees of different children are ordered by their anchors. All items of each child subtree therefore occur consecutively in the resulting total order. Within the child, position

variables again range over $(0, 1)$, and its in-node DAG and interval bounds describe the possible orders inside this block.

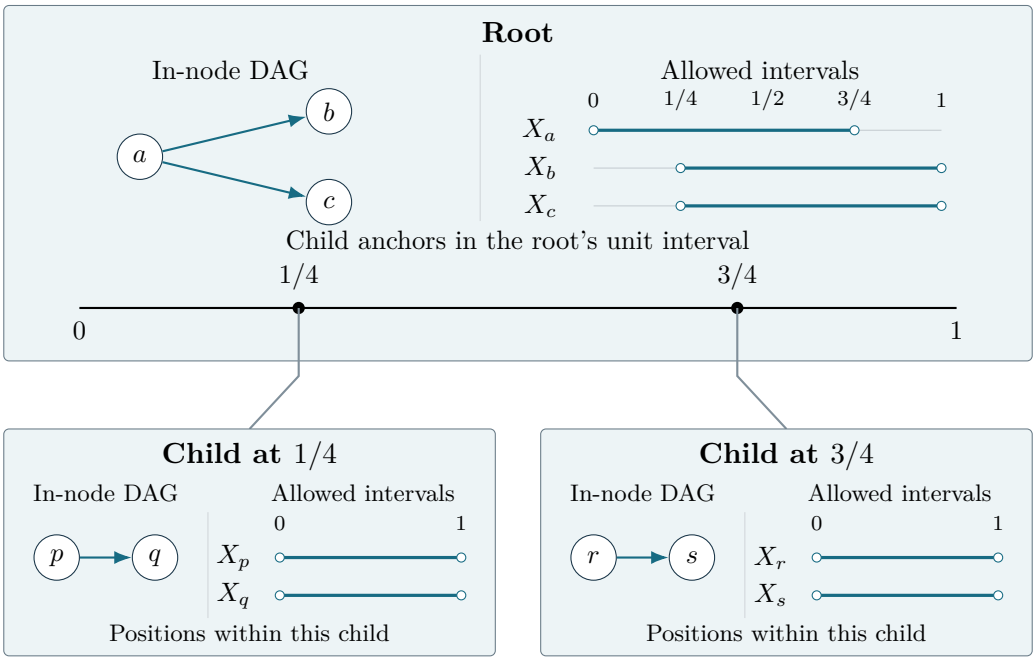



**Figure 4:** A state of the adversary. Each large box is a tree node, containing an in-node DAG and one open interval for each of its items.

In the root of Figure 4, the in-node DAG records $a < b$ and $a < c$. To read an order from this state, choose feasible positions at each node, avoiding ties with other items or child anchors. List the items and anchors from left to right, and recursively replace each anchor by its child's order. For example, root positions $(X_a, X_b, X_c) = (1/3, 1/2, 7/8)$ give

$$p < q < a < b < r < s < c.$$

Other feasible choices give other orders. Every order represented this way must respect all previous answers to the sorter. We call the sequence of these answers the *transcript*.

### 3.2 Answering a comparison

Algorithm 1 shows the algorithm the adversary uses to answer a comparison by the sorter.

To compare $A$ and $B$, start at the root and descend into a child whenever its subtree contains both items. Stop when at least one item is stored at the current node, or when the items lie in different child subtrees. At this node, use an item's free coordinate if it is stored here, and the corresponding child anchor otherwise. In Figure 4, comparing $p$ with $q$ descends into the left child; comparing $p$ with $r$ uses the anchors $1/4$ and $3/4$; and comparing $a$ with $p$ compares $X_a$ with $1/4$ at the root.

Two distinct anchors determine the answer immediately. In the other cases, compute the bounds implied by the in-node DAG. If there is an arc $y \to x$, every lower bound on $X_y$ is also a lower bound on $X_x$; upper bounds propagate in the reverse direction. Write $y \preceq x$ when $y = x$ or a directed path leads from $y$ to $x$. The resulting *effective interval* is

$$\ell_x = \max_{y \preceq x} a_y, \qquad r_x = \min_{y \succeq x} b_y, \qquad W_x = r_x - \ell_x. \tag{1}$$

These endpoints can be computed in two passes through the in-node DAG: lower endpoints in topological order, then upper endpoints in reverse order. We compute them when needed, keeping the record's bounds $a_x, b_x$ as introduced by answers and refinements.

For a free coordinate $X_x$ and an anchor $c$, the answer is determined if $c \le \ell_x$ or $c \ge r_x$. For two free coordinates, a directed path between them determines their order; so do effective intervals that are disjoint or meet only at an endpoint. Return this answer immediately. Lemma 5.2 proves that these tests detect every comparison fixed by the represented orders, including every previously answered pair.

For an undecided comparison, consider the uniform distribution on feasible positions at this node. The subroutine `Certify` computes two nonnegative rational numbers $p_{AB}, p_{BA}$ that are lower bounds on the probabilities of $A < B$ and $B < A$. For a rational tolerance $0 < \eta < 1/32$, it guarantees $p_{AB} + p_{BA} \ge 1 - 2\eta$. Choose the answer with the larger certified fraction, breaking a tie in favor of $A < B$. The chosen answer therefore has probability at least $1/2 - \eta$. For two free items, the events are $X_A < X_B$ and $X_B < X_A$; for a free item and an anchor, they compare its coordinate with the fixed anchor. An answer between two free items adds an arc to the in-node DAG; an answer involving an anchor adds an interval bound.

Before calling `Certify`, we ensure that every queried free coordinate has effective width at least $1/R$, where $R \ge 1$ is a fixed rational threshold. Call such a coordinate *wide*, and a narrower one *narrow*. A narrow queried item is moved into a child, and the query is routed again. This move is called a *refinement*, denoted `Refine`$(x)$; Algorithm 2 in Section 3.3 gives the operation.

We also control the in-node DAGs using integers $k, H \ge 2$. Throughout the interaction, each vertex has at most $k$ neighbors, counting both incoming and outgoing arcs, and each directed path has fewer than $2H$ vertices. Before certifying an undecided comparison, we first remove any path of $H$ vertices by refining its entire list of items. If a queried free item has degree $k$, we refine that item too. Each of these steps restarts the query. When `Certify` is reached, the queried vertices have degree below $k$ and the current node has no $H$-vertex path. Adding one arc therefore preserves the bounds needed throughout the interaction: degrees rise by at most one, and a new path joins two old paths.

The following lemma supplies the guarantee needed by the comparison rule. Section 6 implements `Certify` by counting finite calculations in which enclosing intervals establish one of the two answers.

**Lemma 3.1** (Certified comparison probabilities)**.** *Suppose a node admits a feasible assignment, and its in-node DAG has degree at most $k$ and directed paths of at most $2H$ vertices. For a pair with at least one wide coordinate, or a wide coordinate and a dyadic threshold,* `Certify` *deterministically returns nonnegative rational lower bounds on the probabilities of the two answers. Their sum is at least $1 - 2\eta$.*

**Algorithm 1:** `Compare`$(A, B)$: answer a comparison query.

**Input:** Distinct item labels $A, B$ and the current adversary state.
**Output:** An answer $A < B$ or $B < A$, with the state updated.
**while** *true* **do**
  $v \leftarrow$ the root
  **while** *$A$ and $B$ lie in the same child subtree of $v$* **do**
    $v \leftarrow$ that child
  **if** *$A$ and $B$ lie in different child subtrees of $v$* **then**
    **return** the left-to-right order of their child anchors
  Compute effective intervals at $v$ using (1)
  **if** *effective intervals or a directed path in the in-node DAG at $v$ determine the answer* **then**
    **return** the determined answer
  **if** *the in-node DAG at $v$ has a directed path on $H$ vertices* **then**
    $L \leftarrow$ the list of labels on the selected path
    **foreach** *$x \in L$ in label order* **do**
      `Refine`$(x)$
    **continue**
  **if** *a queried free item has degree $k$ in the in-node DAG at $v$* **then**
    `Refine`(the selected such item)
    **continue**
  **if** *a queried free coordinate is narrow* **then**
    `Refine`(the selected such item)
    **continue**
  $(p_{AB}, p_{BA}) \leftarrow$ `Certify`$(v, A, B, \eta)$
  *answer* $\leftarrow A < B$ if $p_{AB} \geq p_{BA}$, and $B < A$ otherwise
  Store *answer* as an arc in the in-node DAG at $v$ or as an interval bound
  **return** *answer*

Every restart refines at least one item. Its new coordinate in the child is isolated and has interval $(0, 1)$. Further refinements cannot give that coordinate an incident arc or narrow its interval, so it cannot need refinement again during this call. Thus a call makes at most $n$ refinements; Section 5.4 gives the full proof.

For deterministic selection, choose the smallest eligible item label and the lexicographically first eligible directed path. Complete each selected path's list even if earlier refinements break the path.

### 3.3 Moving an item into a child

Algorithm 2 gives `Refine`$(x)$, which moves $x$ from its current node into a child. It computes $x$'s effective interval and calls `ChooseAnchor` to select a position $c$ inside it.

For a narrow coordinate, we choose $c$ in the middle half of its effective interval. For a wide coordinate, the following routine supplies an interval with substantial probability inside and on both sides.

**Lemma 3.2** (A central interval)**.** *Under the hypotheses of Lemma 3.1, for a wide coordinate $x$,* `CentralInterval` *deterministically returns dyadic $u < v$ in $(\ell_x, r_x)$ such that*

$$\mathbb{P}(X_x \leq u) \geq \frac{1}{4}, \qquad \mathbb{P}(u \leq X_x \leq v) \geq \frac{1}{4}, \qquad \mathbb{P}(X_x \geq v) \geq \frac{1}{4}.$$

Section 6 implements this routine using the same enclosing intervals as `Certify`.

We select anchor positions using a fixed infinite binary tree, the *implicit anchor tree*, shown in Figure 5. The same tree is used at every node of the adversary's tree. Each anchor-tree node $u$ has an interval $I(u) = [a, b]$ and a position $c(u) = (a + b)/2$; write $|I(u)| = b - a$. The root has interval $[0, 1]$; the left and right children of $u$ have intervals $[a, c(u)]$ and $[c(u), b]$, respectively.

At depth $m$, the $2^m$ nodes have intervals of length $2^{-m}$ and positions

$$c = (i + \tfrac{1}{2})2^{-m}, \qquad i = 0, \ldots, 2^m - 1. \tag{2}$$

We call $m$ the *anchor depth*, always measured in the anchor tree. These positions are dyadic rationals, so refinement keeps the interval endpoints $a_x, b_x$ dyadic. A finite path from the anchor tree's root specifies an anchor. Algorithm 3 in Section 5.3 gives the rule for choosing a central interval and selecting its shallowest anchor. Equivalently, this is the binary fraction in that interval with the fewest fractional digits.

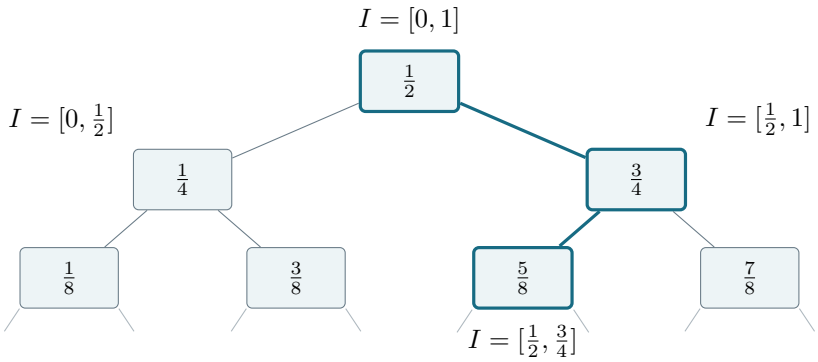



**Figure 5:** The implicit anchor tree. The highlighted path selects 5/8 at depth 2.

Fixing $X_x = c$ places $x$ in the child block anchored at $c$. The arcs incident to $x$ in that in-node DAG become interval bounds: $y \to x$ gives $X_y < c$, and $x \to z$ gives $X_z > c$. Remove $x$ and these arcs from the parent, and retain the new bounds on its neighbors. Insert $x$ into the child anchored at $c$, creating it if needed. In that child, $x$ receives a fresh position variable with interval $(0, 1)$ and no incident arcs; the child's existing constraints remain unchanged. Figure 6 illustrates reuse of an existing child.

**Algorithm 2:** `Refine`$(x)$: move an item into a child.

**Input:** A free item $x$ and the current adversary state.
**Output:** The updated state, with $x$ stored in a child of its former node.
$v \leftarrow$ the node storing $x$
Compute $x$'s effective interval using (1)
$c \leftarrow$ `ChooseAnchor`$(x)$
**foreach** *arc* $y \to x$ *in the in-node DAG at* $v$ **do**
  $b_y \leftarrow \min(b_y, c)$
**foreach** *arc* $x \to z$ *in the in-node DAG at* $v$ **do**
  $a_z \leftarrow \max(a_z, c)$
Remove $x$'s record from $v$ and its vertex and incident arcs from the in-node DAG at $v$
**if** *$v$ has no child anchored at $c$* **then**
  Create a child of $v$ at $c$, with no records and an empty in-node DAG
$u \leftarrow$ the child of $v$ anchored at $c$
Insert a record $(x, X_x, 0, 1)$ at $u$, with a fresh, unassigned position variable $X_x$
Insert $x$ as an isolated vertex in the in-node DAG at $u$

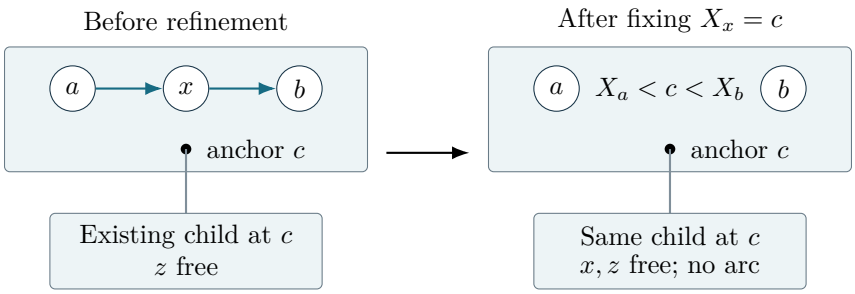



**Figure 6:** The two arc inequalities become bounds on the remaining parent coordinates. The new coordinate of $x$ orders it within the child, where both $x < z$ and $z < x$ remain possible. The anchor $c$ locates the child block among the parent items; the child's constraints determine the orders within that block.

Thus the parent fixes where the child block occurs, while the order of $x$ within that block stays open. Lemma 5.3 proves that this operation preserves all answered comparisons.

Refinement can impose additional order relations. For example, suppose the root contains $A, B, C$ with only $A < B$ recorded. Refining $A$ at $1/2$ restricts $B$ to $(1/2, 1)$. Refining $C$ into the same child then leaves both $A < C < B$ and $C < A < B$ possible, while fixing the additional relation $C < B$. Section 4 shows that we can afford this loss of possible orders while retaining the optimal asymptotic comparison bound.

Table 1 collects the thresholds used by Algorithms 1 and 2 above. In the next section, we explain why the adversary forces any correct sorter to perform many comparisons.

# 4 Why the adversary forces many comparisons

Recall from Algorithm 1 that a larger degree threshold $k$ lets the adversary retain more comparisons involving each item before moving it into a child. Increasing $k$ lets the adversary force a comparison count closer to $n \log n$. The following bound quantifies this dependence.

**Theorem 4.1** (Comparison bound)**.** *For every power of two $k \geq 64$, the adversary of Algorithm 1, with the remaining parameter choices in Table 1, is deterministic and legal, and every call terminates. Every correct sorter makes at least*

$$\frac{n \log n - 3n}{1 + (2 \log k + 17)/k} \tag{3}$$

*comparisons.*

For this section, fix a power of two $k \geq 64$. Section 7 chooses $k$ as a function of $n$ so that it grows without bound while the adversary's running time remains polynomial.

We will bound the number of total orders represented by any state. The bound combines the freedom within nodes with weights assigned to child choices. A single inequality relating refinement cost to volume change will then explain how each of the three refinement rules preserves enough orders. The main proof gives the denominator $1 + O(\log k/k)$; Appendix C verifies the explicit coefficient in (3).

## 4.1 Volume changes

We first measure the freedom left within the individual nodes. At a node $v$, the feasible assignments form the region

$$K_v = \{X : a_x < X_x < b_x \text{ for every free } x, \quad X_x < X_y \text{ for every in-node arc } x \to y\}. \tag{4}$$

Write $V_v = \mathrm{vol}(K_v)$ and take the product over the adversary's tree:

$$Z = \prod_v V_v. \tag{5}$$

Each item has a position variable at exactly one node, so the product region has dimension $n$. A node with no free items contributes volume one. Initially $Z = 1$; whenever every node has positive volume, $0 < Z \le 1$.

Choose the certificate tolerance to be

$$\eta = \frac{1}{100k}. \tag{6}$$

By Lemma 3.1, each chosen answer retains at least $1/2 - \eta$ of its node's volume. Its decrease in $\log Z$ is therefore at most

$$a_\eta = -\log(1/2 - \eta) \le 1 + \frac{1}{2k}. \tag{7}$$

For the inequality, use $2^{-1/(2k)} \le 1 - 1/(4k) \le 1 - 1/(50k) = 1 - 2\eta$. An answer already determined by the stored constraints leaves $Z$ unchanged.

Refinement fixes an item's position in its parent and gives it a fresh unit coordinate in a child. Write

$$g = \log \frac{Z_{\text{after}}}{Z_{\text{before}}} \tag{8}$$

for the resulting change in $\log Z$. We call $g$ the refinement's *volume gain*; it can be negative.

The algorithm must preserve positive node volumes and the bounds needed by the probability routines. It must also return answers consistent with the earlier comparisons. The following proposition gives these guarantees.

**Proposition 4.2** (Consistency and termination)**.** *With the parameter choices of Theorem 4.1, every reached state has positive node volumes and represents only orders consistent with the transcript. Every comparison call terminates and performs at most $n$ refinements. Every in-node DAG has degree at most $k$ and directed paths of fewer than $2H$ vertices throughout the interaction.*

Section 5.4 proves Proposition 4.2.

## 4.2 Counting the orders represented by a state

The product $Z$ measures freedom within individual nodes. For example, two items stored alone in separate children can each retain the full interval $(0, 1)$, giving $Z = 1$, even though the child anchors fix their order. To account for this restriction, assign a positive weight $w(c)$ to each child at anchor $c$, with the weights at each parent summing to at most $1/2$.

For each item, multiply the child weights along the path from the root to its storage node. Let $Q$ be the product of these quantities over all items. An item stored at the root contributes one. Thus each edge weight appears once for every item in its subtree.

**Lemma 4.3** (Number of represented orders)**.** *If the state represents $N_{\text{ord}}$ total orders, then*

$$N_{\text{ord}} \ge n!\, 2^{-n} ZQ. \tag{9}$$

*Proof.* On the fixed state tree, start at the root and enter each existing child $c$ with probability $w(c)$. With the remaining probability, stop and choose a uniform coordinate in $(0, 1)$ at the current node. The stopping probability is at least $1/2$, and the finite tree ensures termination.

Draw one such position independently for every label. The state tree orders these positions by comparing free coordinates and child anchors, then ordering positions within a child recursively.

Ties have probability zero. All labels have the same distribution, so each of the $n!$ relative orders has probability $1/n!$.

Consider the event that every label stops at its actual storage node and the coordinates satisfy all in-node constraints. The required paths have probability $Q$, and stopping at their endpoints contributes at least $2^{-n}$. Conditional on these choices, the coordinates are independent and uniform, so the probability of satisfying the constraints is $Z$. Every order produced by this event is represented by the state. Its probability is therefore at most $N_{\mathrm{ord}}/n!$, proving the lemma. □

Define the cost of choosing a child by

$$\ell(c) = -\log w(c). \tag{10}$$

Records only move downward. Their current paths therefore contain exactly the edge traversals made during refinement, counted once for each moved item. Consequently

$$-\log Q = \sum_{\text{refinements } r} \ell(c_r). \tag{11}$$

### 4.3 One bound for the cost of refinement

We now choose the child weights so that their costs can be compared with the volume changes caused by refinement. For an anchor $c$ at depth $m$ in the implicit anchor tree, set

$$w(c) = \frac{1}{2k}\left(\frac{1-1/k}{2}\right)^m. \tag{12}$$

There are $2^m$ anchors at depth $m$, so their total weight is

$$\sum_c w(c) = \frac{1}{2k}\sum_{m\ge 0}(1-1/k)^m = \frac{1}{2}.$$

In particular, the existing children at any node satisfy the weight condition of Lemma 4.3. The cost is

$$\ell(c) = \gamma m + \log k + 1, \qquad \gamma = 1 - \log(1-1/k). \tag{13}$$

The anchor routine will provide the following guarantee. It says that choosing a child costs approximately the volume gained by fixing the parent coordinate, with an additive term of order $\log k$.

**Lemma 4.4** (Cost and volume change of a refinement)**.** *There are coefficients $a = 1 + O(1/k)$ and $b = O(\log k)$, with $a \ge 1$ and $1 \le b < k+1$, such that every refinement satisfies*

$$g \ge -4, \qquad \ell(c) \le ag + b. \tag{14}$$

*If the refined coordinate is narrow, its gain also satisfies $g > k$.*

Section 5.6 proves the lemma; Appendix C gives explicit choices of $a$ and $b$. The additive term can be absorbed when a refinement has a large gain, or when a batch has a large total child cost. The path rule supplies the latter. Choose

$$H = 2^k. \tag{15}$$

**Lemma 4.5** (Distinct children for a path)**.** *The labels of a complete path batch enter distinct children of their parent in the adversary's tree.*

Section 5.5 proves this property using the order already determined along the selected path.

### 4.4 The comparison bound

*Comparison bound from the refinement guarantee.* Proposition 4.2 gives consistency and termination of every call. The procedures and their tie rules are deterministic. We first explain how each refinement rule accounts for the additive term in (14).

*Narrow coordinates.* Since $g > k$, a narrow refinement satisfies

$$\ell(c) \le ag + b \le (a + b/k)g.$$

*Complete path batches.* Write $L = \sum_i \ell(c_i)$ and $G = \sum_i g_i$ for a batch of $H$ moves. Its children are distinct by Lemma 4.5. Their weights sum to at most $1/2$, so the arithmetic–geometric mean inequality gives

$$\prod_{i=1}^{H} w(c_i) \le \left(\frac{1}{2H}\right)^H, \qquad L \ge H \log(2H) = H(k+1).$$

Summing (14) and substituting $H \le L/(k+1)$ gives

$$L \le aG + bH \le aG + \frac{b}{k+1}L.$$

Thus $L \le \lambda G$, where

$$\lambda = \frac{a}{1 - b/(k+1)} = 1 + O(\log k/k). \tag{16}$$

The assumptions $a, b \ge 1$ give

$$\lambda - a = \frac{ab}{k+1-b} \ge \frac{b}{k},$$

so the same factor $\lambda$ also covers narrow refinements.

*Degree refinements.* Outside a path batch, count a refinement as a degree refinement whenever its vertex has degree $k$, including when its coordinate is also narrow. The bound $g \ge -4$ gives

$$\ell(c) \le \lambda g + \sigma, \qquad \sigma = b + 4(\lambda - a) = O(\log k). \tag{17}$$

After $q$ comparisons, there have been at most $q/k$ degree refinements: each deletes $k$ in-node arcs, each comparison inserts at most one arc, and an inserted arc can be deleted only once. Hence the total refinement cost satisfies

$$\sum_r \ell(c_r) \le \lambda \sum_r g_r + \frac{\sigma}{k}\, q. \tag{18}$$

Each comparison decreases $\log Z$ by at most $a_\eta$, while a refinement changes it by $g_r$. Since initially $Z = 1$,

$$\sum_r g_r \le \log Z + a_\eta q.$$

Combining this with the remaining-order bound (9) and (11) gives

$$\log N_{\mathrm{ord}} \ge \log(n!) - n + (1 - \lambda)\log Z - \left(\lambda a_\eta + \frac{\sigma}{k}\right) q. \tag{19}$$

Here $(1 - \lambda)\log Z \ge 0$, and

$$\lambda a_\eta + \frac{\sigma}{k} = 1 + O(\log k/k).$$

Thus many orders remain until nearly $\log(n!)$ comparisons have been made. At termination $N_{\mathrm{ord}} = 1$, giving

$$q \ge \frac{\log(n!) - n}{1 + O(\log k/k)}.$$

Appendix C verifies that the explicit choices of $a, b$ give $\lambda a_\eta + \sigma/k \le 1 + (2\log k + 17)/k$. Together with $\log(n!) - n \ge n\log n - 3n$, this proves Theorem 4.1. □

The next section proves the refinement and consistency guarantees used above.

# 5 Refinement and its guarantees

We now prove the refinement guarantees used in Section 4: moving an item preserves the answers already given, and the resulting change in volume controls the cost of choosing a child.

## 5.1 Preserving the represented orders

We use the interpretation of the state given in Section 3. Assignments with equal free coordinates, or with a free coordinate equal to an anchor, lie on a finite union of sets of volume zero and are excluded when reading an order. We maintain the following invariant:

> Every represented total order respects every comparison already answered.

The state may discard some orders that still respect the answers. Legality requires only that at least one remains. Positive volume in every node will ensure this.

The invariant must continue to hold when refinement removes an arc from an in-node DAG.

**Lemma 5.1** (Effective intervals and sections)**.** *Replacing interval bounds by their effective endpoints leaves the feasible region unchanged. The region has positive volume if and only if every effective width is positive. When these widths are positive, fixing $X_x = c$ at any $c \in (\ell_x, r_x)$ leaves a positive-volume section in the remaining coordinates.*

*Proof.* If $y \neq x$ precedes $x$ on a directed path, every feasible assignment satisfies $X_x > X_y > a_y$; the stored bound $X_x > a_x$ covers $y = x$. Taking the maximum over these bounds gives $X_x > \ell_x$. The upper-bound argument is the reverse. Thus all bounds given by (1) already hold.

Positive volume requires positive widths. Conversely, suppose all widths are positive. Along an arc $x \to y$, the definitions give $\ell_x \leq \ell_y$ and $r_x \leq r_y$. Choose numbers $0 < \theta_x < 1$ increasing in a topological order, and set

$$X_x = (1 - \theta_x)\ell_x + \theta_x r_x.$$

Increasing $\theta_x$ to $\theta_y$ strictly increases this value because $r_x > \ell_x$. Replacing its endpoints by the larger endpoints $\ell_y, r_y$ cannot decrease it. Hence $X_x < X_y$. All coordinates are inside their intervals and all arc inequalities hold strictly, so a sufficiently small open neighborhood of this assignment is feasible.

Now fix $X_x = c$ and delete its coordinate. The inequalities imposed by its incident arcs become upper bounds $c$ at its in-neighbors and lower bounds $c$ at its out-neighbors. Upper bounds propagate only backward along arcs, and lower bounds only forward. Any lower bound that previously reached a successor through $x$ is superseded by $c > \ell_x$; any upper bound that reached a predecessor through $x$ is superseded by $c < r_x$. Thus the new effective intervals are

$$\begin{cases} (\ell_y, \min(r_y, c)), & y \text{ precedes } x, \\ (\max(\ell_y, c), r_y), & y \text{ succeeds } x, \\ (\ell_y, r_y), & y \text{ is incomparable with } x. \end{cases}$$

The first two cases are disjoint because the in-node DAG is acyclic. A predecessor retains positive width: $r_y > \ell_y$ and $c > \ell_x \geq \ell_y$. A successor does too: $r_y > \ell_y$ and $c < r_x \leq r_y$. All other widths stay positive. The criterion just proved, applied to the remaining in-node DAG, shows that the section has positive volume. □

**Lemma 5.2** (Recognizing determined comparisons)**.** *Suppose every node region has positive volume. At the node reached by routing a query, the anchor, effective-interval, and directed-path tests in Algorithm 1 detect every comparison that holds in all represented orders.*

*Proof.* Distinct child anchors determine their subtrees' order. A free coordinate $X_x$ ranges over its entire effective interval by Lemma 5.1. It can therefore lie on either side of an anchor $c$ exactly when $\ell_x < c < r_x$.

Consider two free items $x, y$. A directed path fixes their order. If they are incomparable in the in-node DAG, we claim that

$$X_x < X_y \text{ is feasible} \quad \iff \quad \ell_x < r_y. \tag{20}$$

Necessity follows from $\ell_x < X_x < X_y < r_y$. For sufficiency, add the arc $x \to y$. This preserves acyclicity. It caps the effective upper endpoints of $x$ and its predecessors by $r_y$, and raises the effective lower endpoints of $y$ and its successors to $\ell_x$. These sets are disjoint, since a common vertex would give a path from $y$ to $x$. Every affected predecessor has lower endpoint at most $\ell_x < r_y$, and every affected successor has upper endpoint at least $r_y > \ell_x$. All effective widths remain positive, so Lemma 5.1 gives a feasible assignment with $X_x < X_y$.

Applying (20) in both directions shows that both answers are possible precisely when the effective intervals overlap. Each feasible local choice extends to a represented total order by choosing feasible assignments in the other nodes and avoiding ties. Hence an undecided local test always leaves both orders possible. □

**Lemma 5.3** (Refinement and child admission)**.** *Suppose the node regions have positive volume and the order invariant holds. Refining $x$ at an interior anchor $c$ preserves both properties, including when the child at $c$ already exists. The transcript implies no relation between $x$ and any label already in that child subtree.*

*Proof.* Consider first an existing child at $c$, and denote its label set by $S$. Choose a feasible assignment in the parent section $X_x = c$, with the other free coordinates pairwise distinct and different from every child anchor. Such an assignment exists by Lemma 5.1. Also choose feasible assignments in the other nodes.

There is a positive distance from $c$ to every other child anchor and every free parent coordinate in this assignment. All incident inequalities at $x$ are strict. Thus, with everything else fixed, $x$ can be moved slightly to either side of $c$ while keeping the old state feasible. The resulting two represented orders place $x$ immediately before or immediately after the entire block $S$. Both orders respect the transcript. For each $y \in S$, one has $x < y$ and the other has $y < x$; neither relation can therefore be implied by the transcript.

Now take any order represented after the refinement. Deleting $x$ from the child leaves an assignment of its old coordinates and hence an old child order. Apply the preceding perturbation using this assignment's parent coordinates and old child coordinates. It gives two feasible old orders that agree with the new one on every label except $x$. The new order inserts $x$ somewhere into their child block. No answered comparison joins $x$ to a label in $S$, so this insertion violates none of them. Relative to every label outside $S \cup \{x\}$, the whole block occupies the old anchor position. The item $x$ is consequently on the same side of each outside label as in either of the two feasible old orders above. Comparisons between other labels are unchanged. The new order therefore respects the entire transcript.

If there was no child at $c$, the same argument uses an empty block: the new singleton child places $x$ where its fixed parent coordinate stood. Finally, the remaining parent region is a positive-volume section, and the child receives an independent coordinate in $(0, 1)$. All node regions remain positive. □

## 5.2 The volume gained by refinement

Refining an item $x$ stored at node $v$ fixes $X_x = c$ in $v$ and gives $x$ a fresh unit coordinate in the child. To measure the volume change, let $S_x(c)$ be the volume of the section of $K_v$ obtained by

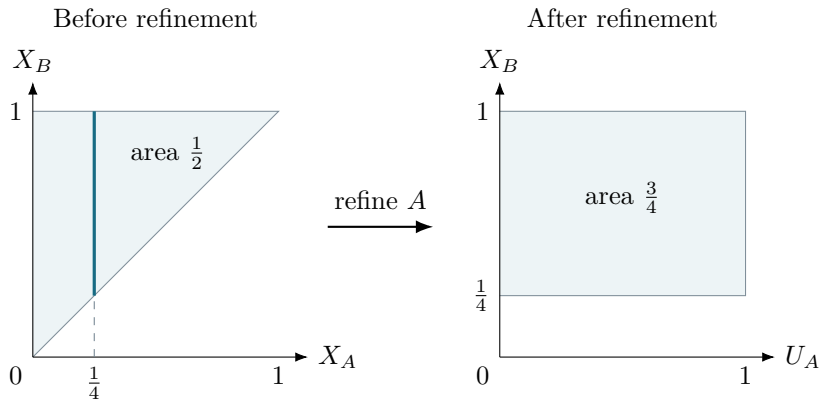



**Figure 7:** The highlighted section of the triangle has length 3/4. Replacing the fixed parent coordinate by a fresh unit coordinate in the child gives the rectangle on the right. Its area is 3/2 times the original area.

fixing $X_x = c$, measured in the remaining coordinates. Under the uniform distribution on $K_v$, the coordinate $X_x$ has density

$$f_x(c) = \frac{S_x(c)}{V_v}, \qquad \mathbb{P}(a < X_x < b) = \int_a^b f_x(t)\,dt.$$

**Lemma 5.4** (Refinement identity)**.** *Suppose every node region has positive volume. Refining $x$ at an interior anchor $c$ satisfies*

$$\frac{Z_{\text{after}}}{Z_{\text{before}}} = f_x(c). \tag{21}$$

*Proof.* Let $u$ be the child at $c$, taking $V_u = 1$ if it is newly created. The parent loses the coordinate $X_x$ and retains precisely the section of volume $S_x(c)$. The child gains a new coordinate $U_x \in (0,1)$, with no constraints involving its old coordinates. Its region becomes $K_u \times (0,1)$ and still has volume $V_u$. Therefore the affected factors in the product change as follows:

$$V_v V_u \quad \longrightarrow \quad S_x(c) V_u.$$

Their ratio is $S_x(c)/V_v = f_x(c)$. All other node factors stay unchanged. The dimension lost in the parent is restored by the new child coordinate. □

Thus the refinement's volume gain is $g = \log f_x(c)$.

For example, the parent region $0 < X_A < X_B < 1$ has area 1/2. Fixing $X_A = 1/4$ leaves the interval $X_B \in (1/4, 1)$ of length 3/4. The new child coordinate $U_A \in (0,1)$ turns this into a rectangle of area 3/4, so refinement multiplies $Z$ by 3/2; see Figure 7. Both regions represent the order $A < B$. Replacing the fixed parent coordinate by a fresh child coordinate gives the larger area.

### 5.3 Choosing an anchor

The choice of anchor balances two quantities: a larger density gives more volume gain, while a smaller anchor depth costs less in (13). We will bound the density using the number of coordinates comparable with $x$. Put

$$B = (k+1)^{2H+1}. \tag{22}$$

In an in-node DAG of degree at most $k$ and directed paths of at most $2H$ vertices, every coordinate comparable with $x$ lies within undirected distance $2H$ of $x$. Thus $B$ bounds the number of other coordinates comparable with $x$.

**Lemma 5.5** (Anchor selection)**.** *Suppose a node's feasible region has positive volume, its interval endpoints are dyadic, and its in-node DAG has degree at most $k$ and directed paths of at most $2H$ vertices. For $R = 2^{2kB}$,* `ChooseAnchor` *deterministically returns an anchor $c \in (\ell_x, r_x)$ at depth $m$ in the implicit anchor tree satisfying*

$$\log f_x(c) \geq (1 - 1/k)m - 4. \tag{23}$$

*If $x$ is narrow, then*

$$\log f_x(c) > k. \tag{24}$$

Algorithm 3 first chooses a closed interval of central positions for $x$, then selects the shallowest anchor in that interval. For a narrow coordinate, we use the central half of its effective interval. For a wide coordinate, `CentralInterval` from Lemma 3.2 supplies an interval with at least a quarter of the probability on each side and at least a quarter inside. We will show that either choice gives the density bound required by Lemma 5.5.

**Algorithm 3:** `ChooseAnchor`$(x)$: choose a position for refinement.

**Input:** A free item $x$ stored at node $v$, its effective interval $(\ell_x, r_x)$, and the fixed width threshold $R$.
**Output:** A dyadic anchor $c \in (\ell_x, r_x)$.

```
W_x ← r_x − ℓ_x
if W_x < 1/R then
    a ← ℓ_x + W_x/4; b ← r_x − W_x/4
else
    [a, b] ← CentralInterval(v, x)
u ← the root of the implicit anchor tree
while true do
    if c(u) < a then
        u ← the right child of u
    else if c(u) > b then
        u ← the left child of u
    else
        return c(u)
```

An anchor at depth $m$ has $m+1$ fractional binary digits in reduced form. Thus the walk finds the shortest binary fraction in $[a, b]$.

**Lemma 5.6** (Shortest anchor in an interval)**.** *For a nondegenerate closed interval $[a, b] \subset (0, 1)$, the walk in Algorithm 3 returns its unique minimum-depth anchor. If its node is $u$ and its depth is $m$, then $[a, b] \subseteq I(u)$ and $|I(u)| = 2^{-m}$.*

*Proof.* Initially $[a, b] \subseteq I(u) = [0, 1]$. Whenever $c(u) < a$, the whole interval lies in the right child's interval; whenever $c(u) > b$, it lies in the left child's interval. The rejected anchor and the other subtree contain no candidate in $[a, b]$. The first accepted anchor therefore has minimum depth, and every other anchor of that depth lies outside $I(u)$. The intervals halve at every step and continue to contain $[a, b]$, so the walk must accept before their length becomes smaller than $b - a$. □

### 5.3.1 Density bounds for central intervals

The first bounds use the effective width and the number of comparable coordinates.

**Lemma 5.7** (Coordinate density bounds)**.** *Suppose a node's feasible region has positive volume, and $x$ has at most $B$ other comparable coordinates and effective interval $(\ell, r)$ of width $W$. Its density satisfies*

$$f_x(t) \le \frac{B+1}{W}, \tag{25}$$

$$f_x(t) \ge \frac{4^{-B}}{W} \qquad (\ell + W/4 \le t \le r - W/4). \tag{26}$$

*Both bounds also hold for the conditional density after fixing coordinates incomparable with $x$, whenever the remaining region has positive volume.*

The lower bound applies to every point of the central half. The upper bound limits the probability in a short interval; it will bound the depth of the anchor chosen for a wide item. Appendix A proves these bounds by comparing convex sections.

For wide items we use another consequence of convexity. A density is *log-concave* if its positive support is convex and its logarithm is concave there. The uniform density on a convex feasible region is log-concave, and Prékopa's marginal theorem gives the same property to each coordinate density [11, Theorem 8]. The following bound says that a position with substantial probability on each side has density comparable to the largest value.

**Lemma 5.8** (Density at central positions)**.** *Let $X$ have a log-concave density $f$. If $\mathbb{P}(X \le c) \ge p$ and $\mathbb{P}(X \ge c) \ge p$ for some $0 < p \le 1/2$, then*

$$f(c) \ge p \sup_t f(t).$$

Appendix A proves the lemma directly from log-concavity.

### 5.3.2 Anchors for wide coordinates

**Lemma 5.9** (Wide anchors)**.** *For a wide item, Algorithm 3 returns an interior anchor $c$ of depth $m$ satisfying*

$$f_x(c) \ge 2^{m-4}, \qquad m \le \log \frac{4(B+1)}{W_x}.$$

*Proof.* Let $J$ be the interval returned by `CentralInterval`, and put $M = \sup_t f_x(t)$. By Lemma 3.2, $J$ is contained in $(\ell_x, r_x)$ and has probability at least $1/4$, so $M|J| \ge 1/4$. Every position $c \in J$ also has probability at least $1/4$ on each side. Lemma 5.8 therefore gives $f_x(c) \ge M/4$ throughout $J$.

Lemma 5.6 selects $c \in J$ at a node $u$ whose interval contains $J$. Its interval has length $2^{-m}$, so

$$2^{-m} f_x(c) \ge |J| \frac{M}{4} \ge \frac{1}{16}.$$

This is the required density bound.

The upper density bound (25) also gives

$$\frac{1}{4} \le \mathbb{P}(X_x \in J) \le \frac{B+1}{W_x}|J| \le \frac{B+1}{W_x}\, 2^{-m}.$$

Rearranging proves the depth bound. □

### 5.3.3 The anchor guarantee

For a wide item, the preceding lemma gives $\log f_x(c) \geq m - 4$. For a narrow item, the central density bound loses $2B$ in its logarithm. We choose

$$R = 2^{2kB}. \tag{27}$$

Then $W_x < 1/R$ makes $\log(1/W_x)$ large enough to absorb this loss.

*Proof of Lemma 5.5.* The in-node DAG bounds imply that $x$ has at most $B$ other comparable coordinates, by (22). For a wide item, Lemma 5.9 gives the stronger lower bound $m - 4$. For a narrow item, write $L = \log(1/W_x)$. The selected anchor is in the central half of its effective interval, so (26) gives

$$\log f_x(c) \geq L - 2B.$$

By Lemma 5.6, its node interval contains that central half. Hence $W_x/2 \leq 2^{-m}$ and $m \leq L + 1$. Since $L > 2kB$,

$$\log f_x(c) \geq L - 2B > (1 - 1/k)L \geq (1 - 1/k)(m - 1) > (1 - 1/k)m - 4.$$

This proves (23). The same density bound also gives $\log f_x(c) > 2B(k - 1) > k$, proving (24). Both cases use a finite deterministic calculation to obtain the central interval, followed by the finite walk of Lemma 5.6. □

The rule also controls the number of bits in a narrow anchor. If the effective endpoints have at most $\Lambda$ fractional binary digits, their midpoint has at most $\Lambda + 1$ and lies in the central half. The chosen anchor has the fewest digits in that interval, so it uses at most $\Lambda + 1$ as well.

## 5.4 Bounds during a comparison call

The preparation loop in Algorithm 1 simplifies the current node before an answer is selected. We check that the in-node DAG bounds hold throughout the call, and that the loop makes only finitely many moves.

**Lemma 5.10** (In-node DAG bounds and termination)**.** *Suppose every in-node DAG has degree at most $k$ and directed paths of fewer than $2H$ vertices at the start of a call. These bounds hold throughout the call. Each label is refined at most once, so the call makes at most $n$ refinements.*

*Proof.* A refinement deletes a vertex and its incident arcs from its parent and adds an isolated vertex to a child. It cannot increase any degree or extend any directed path. Immediately before a new answer is selected, the current in-node DAG has no path on $H$ vertices and the queried free vertices have degree below $k$. Adding their comparison arc leaves degree at most $k$. Any new directed path consists of two old paths joined by this arc, so it has fewer than $2H$ vertices. The certainty tests have already excluded a directed path in either direction between the queried items, so the new arc cannot create a cycle. An answer involving an anchor adds only an interval bound.

Once a label is refined, it has an isolated coordinate with interval $(0, 1)$ in its child. Until the final answer, every subsequent state change is another refinement. These moves affect interval bounds only at neighbors of the vertex being removed, so they cannot narrow this isolated coordinate or give it an incident arc. It cannot lie on a selected path, have degree $k$, or be narrow. Thus it cannot be selected again, even if the query is rerouted to its new node. Every restart refines at least one label, giving the bound of $n$ refinements. □

*Proof of Proposition 4.2.* Initially every node region has positive volume, every order is represented, and the in-node DAG bounds hold. We induct over the changes made by a comparison call. The bounds follow from Lemma 5.10. At a refinement, Lemma 5.5 supplies an interior anchor, and Lemma 5.3 preserves positive volume and the order invariant. This applies to every item of a selected path, including those whose incident arcs have already been removed by earlier moves.

If the local tests determine the answer, it holds in every represented order. Otherwise, the preparation loop ends with wide queried free coordinates. Lemma 3.1 applies to the current positive region; the selected answer retains at least $1/2 - \eta > 0$ of its volume. Recording its arc or interval bound makes that answer true in every represented order and preserves every earlier constraint. This completes the induction.

The comparison calculation and anchor search terminate by Lemmas 3.1 and 5.5. Together with Lemma 5.10, this proves termination of the full call. These analytic lemmas apply to arbitrary positive-volume regions with the stated in-node DAG bounds; their proofs are independent of the adversary's history. □

### 5.5 Distinct children for a path batch

*Proof of Lemma 4.5.* Every in-node arc on the selected path records an answered comparison. The transcript therefore implies an order between every pair of labels on the path. Those implications remain true after the in-node arcs are deleted. By Lemma 5.3, two of these labels cannot be admitted to the same child of their parent. The procedure completes the entire selected batch in this parent, so all its destinations are distinct. □

### 5.6 The refinement guarantee

We finish by relating an anchor's child-choice cost to the volume change it produces. The density guarantee controls its depth, while the chosen weights make the cost almost proportional to that depth.

*Proof of Lemma 4.4.* The refinement identity gives $g = \log f_x(c)$. If the selected anchor has depth $m$, the anchor guarantee (23) implies

$$g \geq -4, \qquad m \leq \frac{g+4}{1-1/k}.$$

Consequently, (13) gives

$$\ell(c) \leq \frac{\gamma}{1-1/k}(g+4) + \log k + 1.$$

Choose any $a \geq \gamma/(1-1/k)$ and put $b = 4a + \log k + 1$. Because $g + 4 \geq 0$, increasing the coefficient to $a$ preserves the inequality, including when $g$ is negative. We obtain $\ell(c) \leq ag + b$. The estimate $\gamma = 1 + O(1/k)$ permits $a = 1 + O(1/k)$ and $b = O(\log k)$. Appendix C supplies explicit choices satisfying $a \geq 1$ and $1 \leq b < k+1$ for every permitted $k$. For a narrow coordinate, $g > k$ is precisely (24). □

## 6 Computing comparison certificates

We implement `Certify` and `CentralInterval` by computing finite lists of enclosing intervals for the queried positions. Disjoint intervals certify a comparison. Counts of intervals lying to the left, inside, and to the right of a proposed central interval certify its three probability bounds.

### 6.1 A coordinate update with finitely many choices

Let $G$ be the in-node DAG. Compute its effective endpoints $\ell_z, r_z$ using the full graph, in temporary storage. For a current assignment $Y$, the interval permitted at coordinate $z$ has endpoints

$$L_z = \max(\{\ell_z\} \cup \{Y_u : u \to z\}), \qquad U_z = \min(\{r_z\} \cup \{Y_u : z \to u\}).$$

Fix a power of two $S$. For a digit $j \in \{0, \dots, S-1\}$, update

$$Y_z \leftarrow \frac{Y_z + (S-1-j)L_z + jU_z}{S}. \tag{28}$$

The coefficients are nonnegative and sum to one. Thus an assignment satisfying the interval bounds and arc inequalities weakly continues to do so after an update, including when $L_z = U_z$.

Choosing the digit uniformly preserves the uniform distribution on the feasible region. Start from a uniform feasible assignment and condition on all coordinates other than $z$. Then $Y_z$ is uniform on $(L_z, U_z)$. Its normalized position $r = (Y_z - L_z)/(U_z - L_z)$ is sent to

$$r' = \frac{r+j}{S}.$$

For a fixed digit, this is uniform on the corresponding one of the $S$ equal subintervals. Averaging over the digits makes $r'$ uniform on $(0,1)$ again.

### 6.2 Enclosing intervals from two calculations

Fix a topological order of the full graph $G$, and let $T$ be the set of queried coordinates. Retain every vertex within undirected distance $\rho$ of $T$, together with all arcs between retained vertices. Copy their effective endpoints from the full graph. Vertices at distance exactly $\rho$ form the boundary; those at smaller distance form the interior.

Run two calculations using the same digits. The lower calculation starts at $A_z = \ell_z$, and the upper calculation starts at $B_z = r_z$. Keep the boundary values fixed throughout both calculations. A *sweep* updates each interior coordinate once, in the restriction of the full topological order. All neighbors of an updated vertex are retained.

Enumerate every assignment of digits to $t$ sweeps. If there are $b'$ interior vertices, this gives

$$N = S^{b't} \tag{29}$$

computations. Each contributes the intervals $[A_x, B_x]$, $x \in T$, to the list, including repeated entries.

**Algorithm 4:** `Enclose`: compute a finite list of enclosing intervals.

**Input:** The retained in-node DAG and its effective bounds, its interior vertices, the restricted full topological order, queried coordinates $T$, and integers $S, t$.
**Output:** A list of enclosing intervals $([A_x, B_x])_{x \in T}$, one per digit assignment.

```
L ← the empty list
foreach assignment of digits j_{z,s} ∈ {0,...,S−1} to the b′t interior visits do
    A_z ← ℓ_z and B_z ← r_z for every retained vertex z
    for s = 1,...,t do
        foreach interior vertex z in the supplied order do
            Apply (28) to A_z, using the current A values and digit j_{z,s}
            Apply (28) to B_z, using the current B values and the same digit
    Append ([A_x, B_x])_{x∈T} to L
return L
```

The intervals have an exact probabilistic interpretation. A *coupling* is a joint distribution with specified marginal distributions; here it joins a uniformly selected list entry to a uniform feasible assignment.

**Theorem 6.1** (Local enclosing intervals)**.** *Let $d, h, r \geq 1$ be integers and let $\delta, \alpha \in (0,1)$ be rational. There are computable parameters $S, t, \rho$ and a bound $E$, depending only on $d, h, r, \delta, \alpha$, with the following property.*

*Suppose $G$ is a finite DAG with dyadic interval bounds, degree at most $d$, directed paths of at most $h$ vertices, and a positive-volume feasible region $K \subseteq (0,1)^{V(G)}$. For any nonempty set $T$ of at most $r$ coordinates, Algorithm 4 returns a list satisfying two guarantees:*

1. *A uniformly selected entry can be coupled with $X$ uniform on $K$ so that $A_x \leq X_x \leq B_x$ for every $x \in T$.*
2. *At most a fraction $\alpha$ of its entries have $\max_{x \in T}(B_x - A_x) > \delta$.*

*The calculation enumerates at most $E$ digit assignments. If the stored bounds have at most $\Lambda$ fractional binary digits, every computed endpoint has at most $\Lambda + \log E$ fractional digits.*

The enclosure guarantee holds for every finite number of sweeps and every choice of radius $\rho \geq 1$. Start a full assignment uniformly on $K$ and update the entire graph with independent uniform digits. Share its digits with the two endpoint calculations at interior vertices. The update is increasing in every coordinate, since maxima, minima, and the nonnegative linear combination in (28) are increasing. Thus it preserves the inequalities $A_z \leq X_z \leq B_z$ at an interior update. At a boundary vertex the full position always remains between its fixed effective endpoints, even as it changes during the full calculation. The full assignment remains uniform, and every interior digit sequence has probability $1/N$. This proves the first guarantee.

To make most intervals narrow, we use the degree and path-length bounds. Following dependencies on newly updated predecessors reaches a source within $h$ vertices, where the lower endpoint is fixed. Most digit choices therefore reduce the interval widths. Bounded degree limits the number of dependencies to consider across successive sweeps. Appendix B proves the width bound and gives explicit parameter choices.

### 6.3 Certifying comparison answers

For two queried coordinates, let $p$ be the fraction of entries with $B_x < A_y$, and let $q$ be the fraction with $B_y < A_x$. The routine `Certify` returns $(p, q)$. For a threshold comparison, use the fixed interval $[c, c]$ for the threshold. A directed path that already orders two queried items gives the exact pair $(1, 0)$ or $(0, 1)$.

Each disjoint pair of intervals certifies the same strict order for the enclosed positions. The enclosure guarantee therefore gives exact lower bounds on the two true probabilities. We next bound the fraction of entries whose intervals overlap, including when they touch at one endpoint.

**Corollary 6.2** (A bound for small differences)**.** *Let $X$ be uniform on a positive-volume node region, and suppose $x$ has at most $B$ other comparable coordinates and effective width $W$. Let $Y$ be a fixed number or $X_y$ for a coordinate $y$ incomparable with $x$. For every interval $I$ of length $a$,*

$$\mathbb{P}(X_x - Y \in I) \leq \frac{(B+1)a}{W}. \tag{30}$$

*Proof.* For $Y = X_y$, condition on $X_y$. Lemma 5.7 bounds the conditional density of $x$ by $(B+1)/W$. Integrating over the translate $I + X_y$ and then averaging over $X_y$ proves the claim. For fixed $Y$, apply the unconditional density bound. □

*Proof of Lemma 3.1.* Computing effective endpoints preserves the positive-volume feasible region by Lemma 5.1. Use Theorem 6.1 with

$$d = k, \qquad h = 2H, \qquad r = 2, \qquad \alpha = \frac{\eta}{2}, \qquad \delta = \frac{\eta}{8(B+1)R}. \tag{31}$$

These same parameters will serve `CentralInterval` below. The two certification conditions are disjoint, so $p, q \geq 0$ and $p + q \leq 1$. Enclosure gives $p \leq \mathbb{P}(X_x < X_y)$ and $q \leq \mathbb{P}(X_y < X_x)$.

For an incomparable pair, choose a wide coordinate $x$. If the computed intervals overlap and both have width at most $\delta$, the enclosed positions differ by at most $2\delta$. Corollary 6.2 consequently bounds the unresolved fraction by

$$1 - p - q \leq \alpha + \mathbb{P}(|X_x - X_y| \leq 2\delta) \leq \alpha + 4(B+1)R\delta = \eta.$$

For a threshold $c$, an unresolved interval contains $c$. If its width is at most $\delta$, its enclosed position is within $\delta$ of $c$. The unresolved fraction is at most

$$\alpha + 2(B+1)R\delta = \frac{3\eta}{4}. \tag{32}$$

Both bounds imply the claimed $p + q \geq 1 - 2\eta$. The counts and their denominators are exact integers, so the routine is deterministic. □

### 6.4 Certifying a central interval

For a wide coordinate $x$, `CentralInterval` runs Algorithm 4 with $T = \{x\}$ and the parameters in (31). Write its intervals as $[A_i, B_i]$, $1 \leq i \leq N$. The chosen $S$ is divisible by four, so $N/4$ is an integer. Sort the upper endpoints and take their $N/4$-th value as $u$. Sort the lower endpoints and take their $3N/4$-th value as $v$. Finally, count the intervals wholly contained in $[u, v]$. The following proof shows that at least $N/4$ intervals pass this test; the routine returns $[u, v]$ with these finite certificates.

*Proof of Lemma 3.2.* At least $N/4$ intervals end at or below $u$, and at least $N/4$ begin at or above $v$. Enclosure certifies the required left and right masses. It remains to bound the number of intervals contained in $[u, v]$.

By (32), at most a fraction $\tau = 3\eta/4$ of the intervals contain any fixed real number. This bound applies in particular to $u$ and $v$. An interval extending below $u$ either ends strictly below $u$ or contains $u$. By the rank defining $u$, fewer than $N/4$ intervals end strictly below it. Thus at most $(1/4 + \tau)N$ intervals extend below $u$. Similarly, at most $N/4$ intervals begin strictly above $v$, so at most $(1/4 + \tau)N$ extend above $v$.

It follows that at least $(1/2 - 2\tau)N \geq N/4$ intervals lie wholly inside $[u, v]$, since $\eta < 1/32$. Enclosure gives mass at least $1/4$ inside this interval. The true distribution is continuous and supported on $(\ell_x, r_x)$. The three positive masses therefore imply $\ell_x < u < v < r_x$. The endpoints are dyadic, and sorting and counting finite lists are deterministic. □

The next section chooses the parameters as a function of $n$ and bounds the work in every comparison call.

## 7 A uniform polynomial-time implementation

For each fixed degree threshold $k$, the preceding sections give a finite deterministic algorithm. Its local calculations depend on $k$, while its interval endpoints depend on the comparison history. We first bound the size of the maintained state, then choose $k$ slowly enough that the local calculations also take polynomial time. Computation is measured on a fixed deterministic multitape Turing machine, with integers represented in binary.

### 7.1 Bounding the state throughout the interaction

The certainty tests in Algorithm 1 recognize every comparison fixed by the represented orders (Lemma 5.2). Since those orders respect all earlier answers, a previously answered pair is recognized before any refinement begins. Such a call returns without changing the state. Every call that changes the state therefore compares a previously unanswered pair, so there are at most $\binom{n}{2}$ such calls, however often the sorter repeats a comparison.

Lemma 5.10 bounds the number of refinements in one call by $n$. Consequently their total number $P$, including those in a call still in progress, satisfies

$$P \le n\binom{n}{2}. \tag{33}$$

Each refinement creates at most one node in the adversary's tree, so that tree has at most $P+1$ nodes. The $n$ item records are distributed among these nodes, and each in-node DAG has degree at most $k$. The algorithm stores this tree, its records, and its in-node DAGs.

We store an anchor as an exact dyadic fraction. An anchor of depth $m$ has $m+1$ fractional binary digits. Comparisons between anchors use these exact values.

For a wide item, Lemma 5.9 bounds its anchor precision by a function of $k$ alone. We may use

$$K_0(k) = \lceil \log(16(B+1)R) \rceil + 2.$$

For a narrow item, suppose the stored endpoints have at most $\Lambda$ fractional digits. The midpoint of its effective interval has at most $\Lambda+1$ digits and lies in the central half. The chosen fraction has the fewest digits there, so it uses at most $\Lambda+1$ digits as well. Effective endpoints are maxima and minima of stored bounds, which add no digits. After $P$ refinements,

$$\Lambda \le K_0(k) + P. \tag{34}$$

### 7.2 An explicit budget for the finite calculations

Use the parameter choices in (31) for the local enclosure theorem. Write $S_k$ for its number of digit choices, $t_k$ for the number of sweeps, and $b_k$ for the bound on retained vertices in (45). The number of digit sequences in any one calculation is at most

$$E_k = S_k^{b_k t_k}. \tag{35}$$

This explicit integer will determine which parameters we can afford.

For powers of two $k = 64, 128, \ldots$, a fixed program writes the complete parameter tuple and $E_k$ in binary. On input size $n$, simulate this program for $n$ steps. Among the tuples it has completed, choose the largest $k$ satisfying

$$E_k \le n. \tag{36}$$

Use these parameters for the entire interaction. If no tuple qualifies, answer all comparisons according to the fixed label order.

The initialization takes polynomial time: the simulation writes at most $n$ bits, and its completed tuples can be parsed and compared within a polynomial bound. For every fixed $k$, the program finishes its tuple in some finite time $T(k)$. Once $n \ge \max\{T(k), E_k\}$, that tuple is available and meets the budget. Hence the chosen $k(n)$ tends to infinity.

The budget controls both the number and the size of the computations. There are at most $n$ digit sequences per certificate calculation. Each sequence uses two endpoint calculations, each with at most $b_k t_k \le \log n$ updates. Each calculation adds at most $b_k t_k \log S_k = \log E_k \le \log n$

fractional bits. The frozen boundary requires no updates. Tallies have at most $\log n + 1$ bits. A central-interval calculation stores at most $n$ intervals, sorts their endpoints, and checks the three certificate counts.

The constants in (44) also give $K_0(k) \le \log E_k$: here $M = B$, and the chosen number of sweeps exceeds $\lceil \log(8(B+1)R/\eta) \rceil$, which is larger than $K_0(k)$. Thus (33) and (34) give $\Lambda = O(n^3)$. Every update uses exact arithmetic on polynomially many bits. The shortest-fraction search examines at most the fractional bit length of an available candidate: the midpoint in the narrow case, or an endpoint of the certified central interval in the wide case. Its integer arithmetic therefore also has polynomial bit cost.

A comparison call makes at most $n$ refinements, each requiring at most one central-interval calculation, followed by at most one call to `Certify`. Routing queries, computing effective endpoints, and testing reachability use polynomial time in the stored state. Directed-path lengths are computed in reverse topological order; if $H > n$, no path can require refinement. All tie rules can be implemented by scanning labels in order. Thus each comparison is answered within one fixed polynomial number of bit operations, independent of the chosen parameters and of the length of the preceding interaction.

*Proof of Theorem 1.1.* Initialization and every comparison call are deterministic and take polynomial time. For all sufficiently large $n$, a parameter tuple qualifies. Theorem 4.1 gives legality and forces

$$q \ge \frac{n \log n - 3n}{1 + (2 \log k(n) + 17)/k(n)} = (1 - o(1)) n \log n,$$

since $k(n) \to \infty$. The fixed-order rule for the remaining input sizes is also deterministic and legal. □

# A Coordinate densities

We prove the density bounds used to choose anchors and to turn coordinate approximation into probability approximation. The first bounds follow by fixing the incomparable coordinates and comparing sections of the remaining convex region. Let $V$ be the set of coordinates, let $N = |V|$, and let $K \subseteq \mathbb{R}^V$ be their feasible region. For a coordinate $x$ and a real number $t$, write

$$K_x(t) = \{(z_i)_{i \ne x} : z \in K,\ z_x = t\}, \qquad S_x(t) = \mathrm{vol}_{N-1} K_x(t).$$

Here $K_x(t)$ is the $(N-1)$-dimensional section obtained by fixing coordinate $x$ at $t$. The uniform distribution on $K$ therefore gives $x$ the density

$$f_x(t) = \frac{S_x(t)}{\mathrm{vol}_N K}.$$

These formulas define $f_x$ pointwise, and we use this definition throughout. If some coordinates have already been fixed, we use the same definitions in the remaining free coordinates. In dimension zero, the volume of the singleton consisting of the empty assignment is one.

*Proof of Lemma 5.7.* We may first fix any prescribed set of coordinates incomparable with $x$, at values leaving a positive-volume slice; write $f_x$ for its coordinate density. Fixing a coordinate turns its incident inequalities into interval bounds. These bounds propagate along directed paths, so fixing an incomparable coordinate leaves $x$'s effective endpoints unchanged. By Lemma 5.1, its projection interval remains $(\ell, r)$. The same argument applies successively as more incomparable coordinates are fixed.

Now fix all the remaining coordinates incomparable with $x$. For each choice leaving a nonempty slice, the remaining feasible region $C$ is open and convex, has dimension $d \le B + 1$, and projects

onto $(\ell, r)$ in coordinate $x$. Write $S(t)$ for its section volume at $x = t$ and $g(t) = S(t)/\operatorname{vol} C$ for the resulting density.

Choose points in $\overline{C}$ whose $x$-coordinates are $\ell$ and $r$. Such points exist because $C$ is bounded. For $\ell < t < s < r$, join every point of the section at $t$ to the point at $r$. Since $C$ is open and convex, the image at $x = s$ lies in the section of $C$ there. Each of its $d-1$ directions is scaled by $(r-s)/(r-t)$, so

$$S(s) \geq \left(\frac{r-s}{r-t}\right)^{d-1} S(t) \qquad (t < s). \tag{37}$$

Joining instead to the point at $\ell$ gives

$$S(s) \geq \left(\frac{s-\ell}{t-\ell}\right)^{d-1} S(t) \qquad (s < t). \tag{38}$$

Normalize these inequalities and integrate over $s$ on both sides of $t$:

$$1 \geq g(t)\left(\int_\ell^t \left(\frac{s-\ell}{t-\ell}\right)^{d-1} ds + \int_t^r \left(\frac{r-s}{r-t}\right)^{d-1} ds\right) = \frac{W g(t)}{d}.$$

Thus $g(t) \leq d/W \leq (B+1)/W$.

For the lower bound, choose $t_0$ with $g(t_0) \geq 1/W$, which exists because $g$ integrates to one over an interval of length $W$. For any $s \in [\ell + W/4, r - W/4]$, the appropriate ratio in (37) or (38) is at least $1/4$. Consequently

$$g(s) \geq 4^{-(d-1)} g(t_0) \geq \frac{4^{-B}}{W}.$$

The case $s = t_0$ satisfies the same bound directly.

Finally, average over the additional incomparable coordinates fixed in the second paragraph. If $\omega$ denotes their values, $C_\omega$ the remaining region, and $g_\omega$ its coordinate density, integration of each section gives the pointwise identity

$$f_x(t) = \frac{\int \operatorname{vol}(C_\omega) g_\omega(t)\, d\omega}{\int \operatorname{vol}(C_\omega)\, d\omega}.$$

Empty slices contribute zero. The bounds on $g_\omega$ are uniform in $\omega$, so they hold for $f_x$ at every $t$. This proves both the unconditional bounds and the bounds after the initial conditioning. □

The coordinate densities above are log-concave pointwise: the uniform density on $K$ is log-concave, and integrating out the other coordinates preserves this property by Prékopa's marginal theorem [11, Theorem 8]. We finish with the density bound used for positions having substantial probability on each side.

*Proof of Lemma 5.8.* For $s \leq c \leq t$, log-concavity gives

$$f(s) f(t) \leq f(c) f(s + t - c).$$

Indeed, $c$ and $s + t - c$ are complementary convex combinations of $s, t$; multiply the two log-concavity inequalities. Fixing $t \geq c$ and integrating over $s \leq c$ gives

$$p f(t) \leq f(c) \int_{-\infty}^{c} f(s + t - c)\, ds \leq f(c).$$

Here the first inequality uses the mass at least $p$ to the left of $c$, and the last uses that a density integrates to one. Fixing $s \leq c$ and integrating over $t \geq c$ instead uses the right-hand mass and gives $p f(s) \leq f(c)$. Taking the supremum over both sides of $c$ proves the claim. □

# B Why the enclosing intervals become narrow

The enclosure part of Theorem 6.1 was proved in Section 6. Here we bound the fraction of digit sequences that leave a wide interval at a queried coordinate. We compare the lower and upper calculations with each other. Their initial widths are at most one, and their boundary values stay fixed. For the analysis, choose each interior digit independently and uniformly; each enumerated sequence then has probability $1/N$.

## B.1 The decrease after one sweep

Write $\Delta_z = B_z - A_z$. Superscripts old and new refer to the start of a sweep and the value after a coordinate's update. For an interior vertex $z$, the shared digit $j_z$ gives

$$\Delta_z^{\text{new}} \le \frac{1}{S}\Delta_z^{\text{old}} + \frac{S-1-j_z}{S}\max_{u\to z}\Delta_u^{\text{new}} + \frac{j_z}{S}\max_{z\to u}\Delta_u^{\text{old}}, \tag{39}$$

with an empty maximum equal to zero. The effective interval bounds in the two calculations are identical.

Let $B_h(z)$ be the vertices at undirected distance at most $h$ from $z$, and suppose they are all interior vertices. Put $D = \max_{u\in B_h(z)}\Delta_u^{\text{old}}$ and $a_u = (S-1-j_u)/S$. At $z$, (39) gives

$$\Delta_z^{\text{new}} \le (1-a_z)D + a_z\max_{u\to z}\Delta_u^{\text{new}}.$$

Follow a predecessor attaining the maximum, and continue backwards to a source. This path has at most $h$ vertices; all its vertices and their successors lie in $B_h(z)$. At the source the predecessor term is zero. Substitution along the path bounds the new width by $(1-\prod_u a_u)D$.

Write $\Delta_z(s)$ for the width after $s$ sweeps. Call the pair $(z,s)$ *bad* if, during sweep $s$, the digit at $z$ or any vertex preceding $z$ on a directed path equals $S-1$. Otherwise all factors $a_u$ in the preceding expansion are at least $1/S$, and there are at most $h$ of them. Put $\beta = 1 - S^{-h}$. Consequently,

$$\Delta_z(s) \le \kappa_z(s)\max_{u\in B_h(z)}\Delta_u(s-1), \qquad \kappa_z(s) = \begin{cases} 1, & (z,s) \text{ is bad,} \\ \beta, & \text{otherwise.} \end{cases} \tag{40}$$

## B.2 Following the dependencies through several sweeps

Both $B_h(z)$ and the set consisting of $z$ and its directed-path predecessors have at most $M$ vertices. Choose

$$M = (d+1)^{h+1}, \qquad S = \text{the least power of two at least } 16M^3. \tag{41}$$

The probability that $(z,s)$ is bad is at most $M/S \le 1/(16M^2)$.

For even $t$, iterating (40) traces sequences $z_t = z, z_{t-1}, \dots, z_0$ with $z_{s-1} \in B_h(z_s)$. There are at most $M^t$ sequences. Retain radius $\rho = ht+1$ around the queried set and freeze its outer boundary. For a queried $z$, every $B_h(z_s)$ used at sweep $s \ge 1$ lies in the interior: its vertices are at distance at most $(t-s+1)h \le th < \rho$ from $z$. Thus every predecessor update used in the expansion is performed. The expansion reaches the initial widths before it reaches a frozen boundary. Since those initial widths are at most one,

$$\Delta_z(t) \le \max_{z_t=z,\, z_{s-1}\in B_h(z_s)} \prod_{s=1}^{t} \kappa_{z_s}(s). \tag{42}$$

For a fixed sequence, the events that its pairs are bad use independent digits from different sweeps. The probability that at least $t/2$ pairs are bad is at most

$$2^t\left(\frac{1}{16M^2}\right)^{t/2} = \left(\frac{1}{2M}\right)^t.$$

A union bound over the at most $M^t$ sequences gives probability at most $2^{-t}$ that any sequence has this many bad pairs. Otherwise every product in (42) has at least $t/2$ factors at most $\beta$. Hence

$$\mathbb{P}(\Delta_z(t) > \beta^{t/2}) \le 2^{-t}. \tag{43}$$

### B.3 Explicit accuracy and computation bounds

To leave width at most $\delta$ at every queried coordinate, except for at most a fraction $\alpha$ of digit sequences, choose

$$t = 2\left(S^h \lceil \log(1/\delta) \rceil + \lceil \log(r/\alpha) \rceil\right), \qquad \rho = ht + 1. \tag{44}$$

The inequality $(1-v)^{-q} \ge 1 + qv$ gives $\beta^{S^h} \le 1/2$. Thus $\beta^{t/2} \le \delta$ and $2^{-t} \le \alpha/r$. Equation (43) and a union bound over the at most $r$ queried coordinates prove the width guarantee of Theorem 6.1. The probability here is exactly the fraction of entries in the enumerated list, counted with multiplicity.

Each radius-$\rho$ neighborhood has at most $1 + d + \cdots + d^\rho \le (d+1)^{\rho+1}$ vertices. We may therefore use

$$b = r(d+1)^{\rho+1}, \qquad E = S^{bt} \tag{45}$$

as bounds on the retained vertices and enumerated digit assignments. All these choices are computable from $d, h, r, \delta, \alpha$ alone.

### B.4 Exact arithmetic

Suppose the effective endpoints have at most $\Lambda$ fractional binary digits. An update takes maxima and minima, forms an integer linear combination, and divides by the power of two $S$. It adds at most $\log S$ fractional digits. If $b'$ interior vertices are updated, each endpoint calculation makes $b't$ updates. The two calculations share digits but use their own values, so the precision in either is at most

$$\Lambda + b't \log S \le \Lambda + \log E. \tag{46}$$

All positions remain in $[0,1]$, so their integer numerators need at most one additional bit. Digits and tallies have at most $\log S$ and $\log E + 1$ bits, respectively. This proves the remaining claims of Theorem 6.1.

## C The explicit comparison coefficient

We verify the choices of $a, b$ in Lemma 4.4 and the coefficient claimed in Theorem 4.1. Write $j = \log k$, so $j$ is an integer at least six. Since

$$-\log(1 - 1/k) \le \frac{1}{(k-1)\ln 2} \le \frac{3}{2k},$$

we may take

$$a = \frac{1 + 3/(2k)}{1 - 1/k} = 1 + \frac{5}{2(k-1)}, \qquad b = 4a + j + 1. \tag{47}$$

These choices satisfy $a \ge \gamma/(1-1/k)$ as required by the refinement proof. They also give $a \ge 1$ and $b \ge 1$.

Put

$$D = k^2 - (j+5)k + j - 6.$$

The inequality $k \geq j + 6$ gives $D > 0$, and $k + 1 - b = D/(k-1) > 0$. Hence $b < k + 1$, and the coefficients (16) and (17) become

$$\lambda = \frac{(k+1)(2k+3)}{2D}, \qquad \sigma = j + 1 + 4\lambda.$$

Using $a_\eta \leq 1 + 1/(2k)$ from (7), direct simplification gives

$$1 + \frac{2j+17}{k} - \left(\lambda\left(1 + \frac{1}{2k}\right) + \frac{\sigma}{k}\right) = \frac{P}{4kD}, \tag{48}$$
$$P = 16k^2 - (4j^2 + 80j + 395)k + 4j^2 + 40j - 411.$$

For $j = 6$ and $k = 64$, $P = 293 > 0$. For $j \geq 7$, the inequality

$$4j^2 + 80j + 395 \leq 16 \cdot 2^j = 16k$$

holds at $j = 7$ and persists as $j$ increases: the polynomial on the left grows by less than a factor of two at each step. Also $4j^2 + 40j - 411 > 0$ in this range. Thus $P > 0$, proving

$$\lambda a_\eta + \frac{\sigma}{k} \leq 1 + \frac{2\log k + 17}{k}.$$

Finally,

$$\log(n!) \geq \int_1^n \log x \, dx = n \log n - \frac{n}{\ln 2} + \frac{1}{\ln 2},$$

and $1 + 1/\ln 2 < 3$ gives $\log(n!) - n \geq n \log n - 3n$. Substitution into (19) at termination yields (3).

# D Parameter reference

Table 1 collects the parameters of the adversary. The last two columns distinguish where a parameter's role is introduced from where its value is chosen and explained. All choices are fixed before the first query. Appendix B gives explicit choices for the digit alphabet, number of sweeps, and neighborhood radius in the local enclosure theorem.

**Table 1:** Parameters and their roles. Section references lead to the explanations in the main text.

| Symbol | Value | Role | Introduced | Choice |
|---|---|---|---|---|
| $k$ | Power of two, $k \geq 64$ | Refine a queried free item when it has this many neighbors. | §3.2 | §4 |
| $H$ | $2^k$ | Number of vertices refined in a selected directed path. | §3.2 | §4.3 |
| $B$ | $(k+1)^{2H+1}$ | Bounds the number of other vertices comparable with one vertex in its in-node DAG. | §5.3 | §5.3 |
| $R$ | $2^{2kB}$ | Sets the width threshold: a coordinate is wide when $W_x \geq 1/R$. | §3.2 | §5.3.3 |
| $\eta$ | $1/(100k)$ | The certified answer fractions sum to at least $1 - 2\eta$. | §3.2 | §4.1 |

## References

[1] Mikhail J. Atallah and S. Rao Kosaraju. *An adversary-based lower bound for sorting.* Information Processing Letters 13(2), 55–57, 1981.

[2] Allan Borodin, Leonidas J. Guibas, Nancy A. Lynch, and Andrew C. Yao. *Efficient searching using partial ordering.* Information Processing Letters 12(2), 71–75, 1981. doi:10.1016/0020-0190(81)90005-3.

[3] Dana Richards and Pravin M. Vaidya. *On the distribution of comparisons in sorting algorithms.* BIT 28, 764–774, 1988. Authors' paper.

[4] Gerth Stølting Brodal, Shiva Chaudhuri, and Jaikumar Radhakrishnan. *The randomized complexity of maintaining the minimum.* Nordic Journal of Computing 3(4), 337–351, 1996. Authors' report, BRICS RS-96-40.

[5] Haim Kaplan, Or Zamir, and Uri Zwick. *The amortized cost of finding the minimum.* Proceedings of the Twenty-Sixth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 757–768, 2015. Published online 22 December 2014. doi:10.1137/1.9781611973730.51. Authors' paper.

[6] Haim Kaplan, Or Zamir, and Uri Zwick. *A sort of an adversary.* Proceedings of the Thirtieth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 1291–1310, 2019. doi:10.1137/1.9781611975482.79.

[7] Richard P. Stanley. *Two poset polytopes.* Discrete & Computational Geometry 1, 9–23, 1986. doi:10.1007/BF02187680. Author's copy.

[8] Graham Brightwell and Peter Winkler. *Counting linear extensions.* Order 8(3), 225–242, 1991. doi:10.1007/BF00383444.

[9] Martin Dyer, Alan Frieze, and Ravi Kannan. *A random polynomial-time algorithm for approximating the volume of convex bodies.* Journal of the ACM 38(1), 1–17, 1991. doi:10.1145/102782.102783. Authors' manuscript.

[10] Jeff Kahn and Jeong Han Kim. *Entropy and sorting.* Journal of Computer and System Sciences 51(3), 390–399, 1995. doi:10.1006/jcss.1995.1077.

[11] András Prékopa. *On logarithmic concave measures and functions.* Acta Scientiarum Mathematicarum 34, 335–343, 1973. Publisher's paper.